

[Uma breve citação para começar]

“Isso aqui vai ser uma **palestra** de **GitHub**
intermediário.”

— Um sábio contemporâneo (2025)

Paráfrase livre de uma lenda que se foi...

2026-09-28

[Capacitação de Git e GitHub]

00:00 — antes de começar

Deixe este slide na tela enquanto a turma chega. Ele quebra o gelo sozinho.

palestra
intermediário **GitHub**
— Um sábio contemporâneo (2025)

Paráfrase livre de uma lenda que se foi...

for_code []

[Capacitação de Git e GitHub]

Do primeiro commit à automação

Ronivaldo D. Andrade - Diretor de Projetos – Edição 2025 · Revisão 2026

2026-09-28

[Capacitação de Git e GitHub]

- []

[Capacitação de Git e GitHub]

Ronivaldo D. Andrade - Diretor de Projetos – Edição 2025 · Revisão 2026

00:00 — Abertura (5 min para o bloco todo)

Boas-vindas. **Faça agora:** peça para todo mundo abrir o navegador em github.com e entrar na conta, ou criar uma. É o que mais atrasa o bloco 1.

[Apresentação]



Ronivaldo D. Andrade

**Estagiário de Análise e Desenvolvimento de Sistemas
na Marinha do Brasil**

Diretor de Projetos da for_code

Matemática Aplicada — UFRJ

linkedin.com/in/ronidomingues · github.com/ronidomingues

2026-09-28

[Capacitação de Git e GitHub]



Estagiário de Análise e Desenvolvimento de Sistemas
na Marinha do Brasil
Diretor de Projetos da for_code
Matemática Aplicada — UFRJ

00:01

Quarenta segundos. O ponto é mostrar que você usa Git e GitHub no trabalho todo dia, e não só em sala.

[O trato de hoje]

Você vai sair daqui sabendo

- Proteger a conta e pedir os benefícios de estudante
- Fazer commits, branches e merges
- Resolver um conflito sem pânico
- Contribuir por fork e Pull Request
- Publicar um site e rodar uma automação

O que NÃO cabe em 2 horas

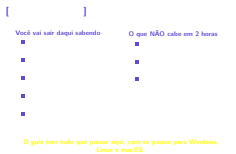
- Rebase e reescrita de histórico a fundo
- Estratégias de branching de times grandes
- Pipelines de CI/CD de produção

O guia tem tudo que passar aqui, com os passos para Windows, Linux e macOS.



2026-09-28

[Capacitação de Git e GitHub]



00:02

Diga que ninguém precisa anotar comandos: está tudo no guia, com os exercícios e as respostas.

[Como as 2 horas vão passar]

Bloco	Assunto	Início	Tempo
0	Abertura	00:00	5 min
1	A conta: 2FA e benefícios de estudante	00:05	15 min
2	Instalação, configuração e login	00:20	20 min
3	Git no dia a dia: commit, branch e conflito	00:40	20 min
—	Pausa	01:00	5 min
4	Colaboração: fork, Pull Request e commits	01:05	20 min
5	GitHub Pages e GitHub Actions	01:25	15 min
6	Git LFS e assinatura de commits	01:40	15 min
7	Encerramento	01:55	5 min



2026-09-28

[Capacitação de Git e GitHub]

Bloco	Assunto	Início	Tempo
0	Abertura	00:00	5 min
1	A conta: 2FA e benefícios de estudante	00:05	15 min
2	Instalação, configuração e login	00:20	20 min
3	Git no dia a dia: commit, branch e conflito	00:40	20 min
—	Pausa	01:00	5 min
4	Colaboração: fork, Pull Request e commits	01:05	20 min
5	GitHub Pages e GitHub Actions	01:25	15 min
6	Git LFS e assinatura de commits	01:40	15 min
7	Encerramento	01:55	5 min

00:03

Combine a pausa em voz alta. **Controle de tempo:** se atrasar, o corte sai do bloco 6 (LFS e assinatura estão completos no guia).

2026-09-28

[Capacitação de Git e GitHub]

BLOCO 1

00:05 · 15 min

BLOCO 1

A conta no GitHub

00:05 · 15 min



[Git não é GitHub]

Git

Programa de controle de versão. Roda no seu computador, sem internet.

```
git commit  
git branch  
git merge
```

GitHub

Plataforma que hospeda repositórios Git e acrescenta colaboração.

Pull Requests · Issues
Pages · Actions · Copilot

O Git é o motor. O GitHub é a garagem com oficina.



2026-09-28

[Capacitação de Git e GitHub]

00:05

A confusão entre os dois é a mais comum. Mostre que o Git funciona sozinho: `git init` e `git commit` não precisam de internet.

Git
Programa de controle de versão. Roda no seu computador, sem internet.
`git commit`
`git branch`
`git merge`

GitHub
Plataforma que hospeda repositórios Git e acrescenta colaboração.
Pull Requests · Issues
Pages · Actions · Copilot

O Git é o motor. O GitHub é a garagem com oficina.

[Proteja a conta: 2FA]

- O GitHub **exige** autenticação em dois fatores de quem contribui com código
- Aviso por e-mail, **45 dias** para ativar, mais 7 de carência
- **Settings** → **Password and authentication** → **Enable 2FA**
- Método principal: **aplicativo autenticador** (TOTP)
- Reserva: passkey ou chave de segurança

Guarde os códigos de recuperação

Eles são a única forma de entrar se você perder o celular.

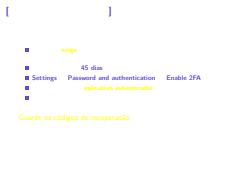


2026-09-28

[Capacitação de Git e GitHub]

00:08

Faça agora: quem já tem conta ativa o 2FA em dois minutos. Quem está criando, faz depois (guia, capítulo 2).



[Benefícios para estudantes]

O que vem

- **GitHub Pro** na conta pessoal
- **Copilot Student**, ativado à parte
- **Student Developer Pack**

GitHub Pro vs Free

- Actions: 3.000 vs 2.000 min/mês*
- Packages: 2 GB vs 500 MB
- Pages em repositório privado
- Suporte por e-mail

* Minutos contados só em repositórios privados. Nos públicos, o Actions é grátis.

Como pedir

1. Cadastre o e-mail acadêmico
2. github.com/settings/education/benefits
3. **Start an application**
4. Anexe o comprovante
5. Aprovado? Ative o Copilot Student



2026-09-28

[Capacitação de Git e GitHub]



00:12

Correção comum: o Copilot **não** é o Pro, é o *Copilot Student*, e a ativação é um passo separado que pode levar dias. E todo mundo tem o *Copilot Free*, mesmo sem ser estudante.

Dica pessoal: a foto da carteirinha com validade é a mais rápida de aprovar.

2026-09-28

[Capacitação de Git e GitHub]

BLOCO 2

00:20 · 20 min

BLOCO 2

Instalação, configuração e login

00:20 · 20 min



[Instalando o Git e o GitHub CLI]

Windows (PowerShell)

```
winget install --id Git.Git -e --source winget  
winget install --id GitHub.cli
```

Linux (Debian e Ubuntu)

```
sudo apt install git git-lfs      # e o gh pelo repositório  
                                oficial (guia, cap. 3)
```

macOS (Homebrew)

```
brew install git git-lfs gh
```

No Windows, daqui para frente, use o Git Bash.



2026-09-28

[Capacitação de Git e GitHub]

Windows (PowerShell)
winget install --id Git.Git -e --source winget
winget install --id GitHub.cli
Linux (Debian e Ubuntu)
sudo apt install git git-lfs # e o gh pelo repositório
 oficial (guia, cap. 3)
macOS (Homebrew)
brew install git git-lfs gh
No Windows, daqui para frente, use o Git Bash.

00:20

O comando do **gh** no Linux é longo: mande copiar do guia, não do slide. **Faça agora:** todo mundo instala. Circule pela sala.

[Dizendo ao Git quem é você]

```
git config --global user.name "Seu Nome"
git config --global user.email "seu.email@exemplo.com"
git config --global init.defaultBranch main
git config --global pull.rebase false
```

- O e-mail deve estar cadastrado na conta do GitHub
- Quer privacidade? Use o e-mail **noreply** do GitHub (**Settings** → **Emails**)
- O **pull.rebase false** evita o erro de *divergent branches* no primeiro conflito



2026-09-28

[Capacitação de Git e GitHub]

```
git config --global user.name "Seu Nome"
git config --global user.email "seu.email@exemplo.com"
git config --global init.defaultBranch main
git config --global pull.rebase false
```

Emails noreply Settings

pull.rebase false

00:28

Peça para todos conferirem com `git config --global --list`.

[Login: o gh resolve]

```
gh auth login
```

Where do you use GitHub?	GitHub.com
Preferred protocol for Git?	HTTPS
Authenticate Git with your GitHub credentials?	Yes
How would you like to authenticate?	Login with a web browser

Por que isso importa

Desde 2021 o GitHub não aceita a senha no terminal. O **Yes** empresta o login do gh ao Git: é isso que faz o `git push` funcionar.



2026-09-28

[Capacitação de Git e GitHub]

00:32

Confira com `gh auth status`. Quem preferir SSH tem o passo a passo no guia (capítulo 4), junto com os tokens fine-grained.

git auth login

Where do you use GitHub?	github.com
Preferred protocol for Git?	https
Authenticate Git with your GitHub credentials?	yes
How would you like to authenticate?	Login with a web browser

Por que isso importa

2026-09-28

[Capacitação de Git e GitHub]

BLOCO 3

00:40 · 20 min

BLOCO 3

Git no dia a dia

00:40 · 20 min



[As três áreas do Git]



Commit é uma fotografia do projeto, com autor, data e o porquê.



2026-09-28

[Capacitação de Git e GitHub]



Commit é uma fotografia do projeto, com autor, data e o porquê.

00:40

Analogia: você separa numa caixa o que vai enviar (`add`) e lacra e registra no protocolo (`commit`).

[O primeiro repositório]

```
gh repo create meu-primeiro-repo --public --clone
cd meu-primeiro-repo
echo "# Meu Primeiro Projeto" > README.md
git status
git add README.md
git commit -m "docs: cria o README"
git push -u origin main
```

O `-u` liga a sua main à do GitHub: depois, basta `git push`.



2026-09-28

[Capacitação de Git e GitHub]

```
gh repo create meu-primeiro-repo --public --clone
cd meu-primeiro-repo
echo "# Meu Primeiro Projeto" > README.md
git status
git add README.md
git commit -m "docs: cria o README"
git push -u origin main
```

O `-u` liga a sua main à do GitHub: depois, basta `git push`.

00:44

Faça agora: Exercício 2 do guia. Quem terminar, abre a página do repositório e vê o README formatado.

```
git switch -c feature/login      # cria a branch e entra nela
# ... edita, add, commit ...
git switch main                 # volta para a main
git merge feature/login         # traz o trabalho
```

- Branch é uma **linha de trabalho**, não uma cópia do projeto
- Criar uma branch é instantâneo, mesmo num projeto enorme
- **git switch** é o comando moderno; o **checkout** antigo ainda funciona



2026-09-28

[Capacitação de Git e GitHub]

```
git switch -c feature/login      # cria a branch e entra nela
# ... edita, add, commit ...
git switch main                 # volta para a main
git merge feature/login         # traz o trabalho
```

linha de trabalho

git switch checkout

00:50

Desenhe no quadro: a main como uma linha, a branch saindo dela e voltando no merge.

[Conflito não é erro]

Duas pessoas mudaram a **mesma linha**. O Git pede uma decisão:

```
<<<<<<< HEAD
# Meu Primeiro Projeto (editado no computador)
=====
# Meu Primeiro Projeto (editado no site)
>>>>>>> 15261e4
```

1. Deixe só o que deve ficar e apague os marcadores
2. `git add README.md`
3. `git commit` e `git push`

Perdeu-se? `git merge --abort` volta ao estado de antes.



2026-09-28

[Capacitação de Git e GitHub]

00:54

Faça agora: Exercício 4 do guia. Ele cria o conflito de propósito: edita a mesma linha no site e no computador.

Duas pessoas mudaram a **mesma linha**. O Git pede uma decisão:

```
<<<<<<< HEAD
# Meu Primeiro Projeto (editado no computador)
=====
# Meu Primeiro Projeto (editado no site)
>>>>>>> 15261e4
```

1.
2. `git add README.md`
3. `git commit` `git push`

Perdeu-se? `git merge --abort` volta ao estado de antes.

Pausa

01:00 · 5 min



2026-09-28

[Capacitação de Git e GitHub]

01:00 · 5 min

01:00

Use a pausa para ajudar quem ficou para trás na instalação ou no login.

BLOCO 4

Colaboração

01:05 · 20 min



2026-09-28

[Capacitação de Git e GitHub]

BLOCO 4

01:05 · 20 min

[Fork e Pull Request]

1. **Fork**: sua cópia do projeto de outra pessoa
2. **Clone** do seu fork
3. **Branch** para a sua mudança
4. **Commit** e **push** para o seu fork
5. **Pull Request**: o pedido para integrar no projeto original
6. **Revisão**: comentários, ajustes, aprovação e merge

O Pull Request é uma conversa antes de integrar.



2026-09-28

[Capacitação de Git e GitHub]

1. Fork
2. Clone
3. Branch
4. Commit push
5. Pull Request
6. Revisão

O Pull Request é uma conversa antes de integrar.

01:05

Mostre um PR real, com comentários de revisão, no navegador.

[Mão na massa: a lista de presença]

```
gh repo fork ronidomingues/github-training --clone
cd github-training
git switch -c presenca/seu-nome
echo "Minha avaliação" > "presences/Seu Nome.txt"
git add presences
git commit -m "chore(presences): registra a presença de Seu
Nome"
git push -u origin presenca/seu-nome
gh pr create --repo ronidomingues/github-training --fill
```

Aceito o PR, o seu nome entra no anexo do guia.



2026-09-28

[Capacitação de Git e GitHub]

```
gh repo fork ronidomingues/github-training --clone
cd github-training
git switch -c presenca/seu-nome
echo "Minha avaliação" > "presences/Seu Nome.txt"
git add presences
git commit -m "chore(presences): registra a presença de Seu
Nome"
git push -u origin presenca/seu-nome
gh pr create --repo ronidomingues/github-training --fill
```

Aceito o PR, o seu nome entra no anexo do guia.

01:10

Exercício 7 do guia. Aceite alguns PRs ao vivo, para mostrarem o merge acontecendo.

[Mensagens que se explicam]

Conventional Commits: <tipo>(escopo): descrição

```
feat(relatorio): exporta em PDF
fix(login): aceita senha vazia sem erro
docs(readme): explica a instalação
ci: roda os testes a cada push
```

feat novo recurso	fix correção	docs documentação	test testes
refactor reorganização	style formatação	ci automação	chore manutenção

O melhor padrão é o que a equipe combinou e segue.



2026-09-28

[Capacitação de Git e GitHub]

Conventional Commits: <tipo>(escopo): descrição

```
feat(relatorio): exporta em PDF
fix(login): aceita senha vazia sem erro
docs(readme): explica a instalação
ci: roda os testes a cada push
```

feat: novo recurso fix: correção docs: documentação test: testes
refactor: reorganização style: formatação ci: automação chore: manutenção

O melhor padrão é o que a equipe combinou e segue.

01:18

Emojis (gitmoji) são opcionais. O guia, capítulo 7, mostra os cuidados: o emoji antes do tipo quebra o padrão para as ferramentas.

2026-09-28

[Capacitação de Git e GitHub]

BLOCO 5

01:25 · 15 min

BLOCO 5

GitHub Pages e GitHub Actions

01:25 · 15 min



[Um site a partir do repositório]

- Sites estáticos: HTML, CSS e JavaScript
- Endereço: `<usuario>.github.io/<repositorio>/`
- **Settings** → **Pages** → **Source**:
 - **Deploy from a branch**: branch `main`, pasta `/(root)`
 - **GitHub Actions**: quando o site é gerado antes de publicar

O erro 404 mais comum

O `index.html` precisa estar na raiz da pasta publicada, e não dentro de uma subpasta.



2026-09-28

[Capacitação de Git e GitHub]



01:25

Exercício 5: o jogo da memória. Mostre o site publicado de um aluno.

[GitHub Actions em um arquivo]

```
on:
  push:
    branches: [ main ]
permissions:
  contents: write          # sem isto, o git push dá 403
jobs:
  executar:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7
      - uses: actions/setup-python@v7
      - run: python main.py
```

.github/workflows/python.yml — exemplo completo no guia, capítulo 8



2026-09-28

[Capacitação de Git e GitHub]

```
on:
  push:
    branches: [ main ]
permissions:
  contents: write          # sem isto, o git push dá 403
jobs:
  executar:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7
      - uses: actions/setup-python@v7
      - run: python main.py
```

.github/workflows/python.yml — exemplo completo no guia, capítulo 8

01:32

Vocabulário: evento, job, step, runner. E o mito do `[skip ci]`: o push feito pelo próprio workflow com o GITHUB_TOKEN **não** dispara o workflow de novo.

2026-09-28

[Capacitação de Git e GitHub]

BLOCO 6

01:40 · 15 min

BLOCO 6

Arquivos grandes e assinatura

01:40 · 15 min



[Git LFS: arquivos grandes]

```
git lfs install                # uma vez por usuário
git lfs track "*.psd"          # grava a regra no .gitattributes
git add .gitattributes arte.psd
git commit -m "chore: versiona a arte com Git LFS"
```

- O GitHub **recusa arquivos acima de 100 MiB** num push comum
- No repositório fica só um ponteiro; o arquivo vai para o LFS
- 10 GiB de armazenamento e 10 GiB de banda por mês



2026-09-28

[Capacitação de Git e GitHub]

```
git lfs install                # uma vez por usuário
git lfs track "*.psd"          # grava a regra no .gitattributes
git add .gitattributes arte.psd
git commit -m "chore: versiona a arte com Git LFS"

recusa arquivos acima de 100 MiB
```

01:40

O limite de 100 MiB é do GitHub, não do Git. Configure o LFS antes de adicionar o arquivo grande.

[Quem fez este commit?]

O autor de um commit é só um texto. Qualquer um pode escrever qualquer nome.

Assinatura com GPG

- `gpg --full-generate-key`
- Chave em **New GPG key**
- `commit.gpgsign true`

Assinatura com SSH

- `gpg.format ssh`
- Chave como **Signing Key**
- Git 2.34 ou mais novo

Resultado: o selo Verified ao lado do commit.



2026-09-28

[Capacitação de Git e GitHub]

01:47

Demonstre com `git -c user.name="Linus Torvalds" commit` como é fácil se passar por outra pessoa. O passo a passo completo está no guia, capítulo 9.



2026-09-28

[Capacitação de Git e GitHub]

BLOCO 7

01:55 · 5 min

BLOCO 7

Encerramento

01:55 · 5 min



[Os erros que você ainda vai ver]

Mensagem

! [rejected] (fetch first)
divergent branches

CONFLICT (content)

Authentication failed

Push de workflow com erro 403

Site do Pages com 404

O que fazer

git pull, resolva e git push

git config --global pull.rebase
false

edite, git add, git commit

gh auth login ou gh auth setup-git

permissions: contents: write

index.html fora da raiz



2026-09-28

[Capacitação de Git e GitHub]

01:55

Não leia a tabela. Diga que todos estão no guia, com causa e correção.

Mensagem
! [rejected] (fetch first)
divergent branches
false
CONFLICT (content)
Authentication failed
Push de workflow com erro 403
Site do Pages com 404

O que fazer
git pull, resolva e git push
git config --global pull.rebase
false
edite, git add, git commit
gh auth login ou gh auth setup-git
permissions: contents: write
index.html fora da raiz

[Para onde ir depois]

Estudar

Pro Git (gratuito, em português)

GitHub Skills

Learn Git Branching

Praticar

Um projeto seu com README,
licença e workflow



Guia e materiais

andradasdev.github.io/github



2026-09-28

[Capacitação de Git e GitHub]

01:57

Passe o link. O guia tem os exercícios com respostas: ninguém precisava ter anotado nada.

Estudar

Praticar



andradasdev.github.io/github

for_code[]

Obrigado!

Capacitação de Git e GitHub

Ronivaldo D. Andrade

Edição 2025 · Revisão 2026



2026-09-28

[Capacitação de Git e GitHub]

- []

Obrigado!

Capacitação de Git e GitHub
Ronivaldo D. Andrade
Edição 2025 · Revisão 2026



02:00

Abra para perguntas. Lembre de aceitar os PRs de presença que ficaram abertos.